

Research Pipeline: Data Loading, Sampling, Preprocessing, and LIME Attribution

This notebook loads SNLI dataset, samples subsets, preprocesses them for attribution, and runs LIME explanations on SNLI examples.

Done for 300 samples of snli without stop words

PART 1: Setup environment and data

Import necessary libraries

```
In [ ]: import os
import json
import time
import random
import torch
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter, defaultdict
import pandas as pd
from datasets import load_dataset, load_from_disk
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from lime.lime_text import LimeTextExplainer
from tqdm import tqdm
import nltk
try:
    stopwords.words("english")
except LookupError:
    nltk.download("stopwords")
from nltk.corpus import stopwords
import warnings
import datetime
import subprocess
from sklearn.metrics import confusion_matrix
import argparse

# Silence noisy logs
warnings.filterwarnings('ignore')
import transformers
transformers.logging.set_verbosity_error()
```

```
In [ ]: # === Labels (single source of truth) ===
# SNLI dataset ground-truth indices:
# 0 = entailment, 1 = neutral, 2 = contradiction
SNLI_LABELS = ['entailment', 'neutral', 'contradiction']

# roberta-large-mnli logits/probability indices:
# 0 = contradiction, 1 = neutral, 2 = entailment
```

```

MNLI_LABELS = ['contradiction', 'neutral', 'entailment']

# useful maps for when you must compare integers
SNLI_TO_MNLI_IDX = {SNLI_LABELS.index(name): MNLI_LABELS.index(name) for name in SNLI_LABELS}
MNLI_TO_SNLI_IDX = {v: k for k, v in SNLI_TO_MNLI_IDX.items()}

SCHEMA_VERSION = "1.1" # single source of truth

# Set reproducible seeds
def set_all_seeds(seed=42):
    """Set all random seeds for reproducibility"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    print(f"✅ All seeds set to {seed}")

def get_git_hash():
    """Get current git hash for reproducibility"""
    try:
        git_hash = subprocess.check_output(['git', 'rev-parse', 'HEAD'],
                                           stderr=subprocess.DEVNULL).decode('ascii').strip()
        return git_hash[:8] # Short hash
    except Exception:
        return "no-git"

def save_run_config(dirs, **kwargs):
    """Save run configuration for reproducibility"""
    config = {
        "schema_version": SCHEMA_VERSION,
        "timestamp": datetime.datetime.now().isoformat(),
        "git_hash": get_git_hash(),
        "model_name": "roberta-large-mnli",
        "random_seed": 42,
        "num_samples": kwargs.get('num_samples', 50),
        "chunk_size": kwargs.get('chunk_size', 64),
        "lime_num_samples": kwargs.get('lime_num_samples', 1000),
        "lime_num_features": 10,
        "k_tokens": 3,
        "snli_labels": SNLI_LABELS,
        "mnli_labels": MNLI_LABELS,
        "snli_to_mnli_idx": SNLI_TO_MNLI_IDX,
    }

    config_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "run_config.json")
    write_json(config_path, config)
    print(f"✅ Run config saved to: {config_path}")
    return config

def _to_py(o):
    if isinstance(o, dict):
        return { _to_py(k): _to_py(v) for k, v in o.items() }
    if isinstance(o, (list, tuple)):
        return [ _to_py(v) for v in o ]
    if isinstance(o, (np.integer,)):
        return int(o)
    if isinstance(o, (np.floating,)):
        return float(o)

```

```

if isinstance(o, np.ndarray):
    return o.tolist()
return o # fall back

def ensure_dir(path: str):
    os.makedirs(path, exist_ok=True)

def file_nonempty(path: str) -> bool:
    return os.path.isfile(path) and os.path.getsize(path) > 0

def read_json(path: str):
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def write_json(path: str, data):
    data = with_schema(data)
    ensure_dir(os.path.dirname(path))
    with open(path, "w", encoding="utf-8") as f:
        json.dump(_to_py(data), f, indent=2, ensure_ascii=False)

def with_schema(d):
    # attach schema unless the object already has it
    if isinstance(d, dict) and "schema_version" not in d:
        d = {"schema_version": SCHEMA_VERSION, **d}
    return d

```

Set Up Data Paths & Directories

```

In [3]: def setup_directories():
        """Setup all required directories"""
        CWD = os.getcwd()
        DATA_DIR = os.path.join(CWD, "data")

        directories = {
            'DATA_DIR': DATA_DIR,
            'CACHE_DIR': os.path.join(DATA_DIR, "hf_cache"),
            'SNLI_LOCAL_DIR': os.path.join(DATA_DIR, "snli"),
            'SAMPLE_SNLI_DIR': os.path.join(DATA_DIR, "sampled_snli_data"),
            'PROCESSED_SNLI_DIR': os.path.join(DATA_DIR, "processed_snli_data"),
            'LIME_OUTPUT_DIR': os.path.join(DATA_DIR, "lime_outputs"),
        }

        # Create all directories
        for dir_path in directories.values():
            os.makedirs(dir_path, exist_ok=True)

        # Create plots subdirectory
        os.makedirs(os.path.join(directories['LIME_OUTPUT_DIR'], "plots"), exist_ok=True)

        return directories

```

Load Datasets and preview (with Local Cache)

Load SNLI dataset from disk if available, otherwise download and cache them locally.

```

In [4]: def load_snli_dataset_fixed(dirs):
        """Load SNLI dataset with proper error handling"""

```

```

print("📁 Loading SNLI dataset...")

dataset_path = dirs['SNLI_LOCAL_DIR']
dataset_ready = os.path.exists(os.path.join(dataset_path, "dataset_dict.json")) # or "st

if dataset_ready:
    print("Loading from local cache...")
    snli_data = load_from_disk(dataset_path)
else:
    print("Downloading SNLI dataset...")
    snli_data = load_dataset("snli", cache_dir=dirs['CACHE_DIR'])
    snli_data.save_to_disk(dataset_path)

# Validate dataset
if 'train' not in snli_data:
    raise ValueError("SNLI dataset does not contain 'train' split.")

sample = snli_data['train'][0]
required_fields = ['premise', 'hypothesis', 'label']
for field in required_fields:
    if field not in sample:
        raise ValueError(f"SNLI dataset missing field: {field}")

print(f"✅ SNLI dataset loaded: {len(snli_data['train'])} training examples")
print("Sample:", sample)

return snli_data

```

Sample 300 Examples from Each Dataset and Save to disk

Randomly sample 300 valid examples from SNLI and CommonsenseQA for fast experimentation.

```

In [ ]: def sample_snli_dataset_fixed(dataset, dirs, num_samples=300):
    """Ensure we have at least `num_samples` unique, valid items; extend if needed."""
    sample_json_path = os.path.join(dirs['SAMPLE_SNLI_DIR'], "snli_sample.json")

    # start from existing
    existing = []
    if file_nonempty(sample_json_path):
        print(f"📄 Found existing SNLI sample at {sample_json_path}; loading...")
        existing = read_json(sample_json_path)

    have = { (e["premise"], e["hypothesis"]) for e in existing }
    target = int(num_samples)

    if len(existing) >= target:
        print(f"✅ Already have {len(existing)} samples (>= {target}); reusing.")
        return existing

    print(f"🔗 Need {target} samples; currently {len(existing)}. Extending...")
    # collect additional, unique, valid examples
    valid_indices = [i for i, ex in enumerate(dataset['train']) if ex['label'] != -1]
    random.seed(42)
    random.shuffle(valid_indices)

    next_id = max([e.get("id", -1) for e in existing] + [-1]) + 1
    for idx in valid_indices:
        if len(existing) >= target:

```

```
        break
    ex = dataset['train'][idx]
    key = (ex['premise'], ex['hypothesis'])
    if key in have:
        continue
    existing.append({
        "id": next_id,
        "premise": ex["premise"],
        "hypothesis": ex["hypothesis"],
        "label": int(ex["label"]),
        "label_name": SNLI_LABELS[int(ex["label"])]
    })
    have.add(key); next_id += 1

write_json(sample_json_path, with_schema({"data": existing}))
print(f"✅ Saved {len(existing)} samples to: {sample_json_path}")
return existing
```

Preprocess Sampled Data for Attribution

Convert SNLI and CSQA samples into model-ready format and save for later use in LIME/SHAP or other explainers.

```
In [6]: def preprocess_snli_for_roberta(snli_sample_data, dirs):
        """
        Preprocess SNLI data specifically for RoBERTa-MNLI format
        Preprocess once; if processed file exists, load and return.
        """
        processed_path = os.path.join(dirs['PROCESSED_SNLI_DIR'], "processed_snli.json")
        if file_nonempty(processed_path):
            print(f"🔍 Found existing processed SNLI at {processed_path}; loading...")
            return read_json(processed_path)

        print(f"🔄 Preprocessing SNLI data for RoBERTa-MNLI...")

        processed_snli = []
        for ex in snli_sample_data:
            # RoBERTa format: premise</s></s>hypothesis
            # But tokenizer handles this automatically, so we just use: premise<sep>hypothesis
            text = f"{ex['premise']}</s></s>{ex['hypothesis']}"

            processed_entry = {
                "id": ex["id"],
                "input_text": text,
                "premise": ex["premise"],
                "hypothesis": ex["hypothesis"],
                "label": ex["label"],
                "label_name": ex["label_name"],
                "dataset": "snli"
            }
            processed_snli.append(processed_entry)

        # Save processed data
        write_json(processed_path, processed_snli)

        print(f"✅ Saved {len(processed_snli)} processed examples to: {processed_path}")

        return processed_snli
```

PART 2: ROBERTA-MNLI MODEL SETUP

```
In [ ]: class RoBERTaMNLIClassifier:
        """Proper RoBERTa-MNLI classifier for LIME explanations"""

        def __init__(self, model_name="roberta-large-mnli", use_fp16=True):
            print(f"📦 Loading {model_name} model...")

            self.model_name = model_name
            self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
            print(f"Using device: {self.device}")

            # Load model and tokenizer
            self.tokenizer = AutoTokenizer.from_pretrained(model_name)
            self.model = AutoModelForSequenceClassification.from_pretrained(model_name)
            if use_fp16 and self.device.type == "cuda":
                self.model.half()
            self.model.to(self.device)
            self.model.eval()

            # Label mapping for MNLI (RoBERTa uses different order than SNLI)
            self.label_mapping = {i: name for i, name in enumerate(MNLI_LABELS)}
```

```

print("✅ Model loaded successfully!")
self._test_model()

def _test_model(self):
    """Test model with a simple example"""
    print("🧪 Testing model...")

    test_premise = "The cat is sleeping on the couch."
    test_hypothesis = "The cat is awake."
    test_text = f"{test_premise}</s>{test_hypothesis}"

    probs = self.predict_proba([test_text])[0]
    predicted_label = int(np.argmax(probs))
    confidence = float(np.max(probs))

    print(f"Test input: '{test_premise}' vs '{test_hypothesis}'")
    print(f"Probabilities: {probs}")
    print(f"Predicted: {self.label_mapping[predicted_label]} (confidence: {confidence:.4f})

    # Should predict contradiction with high confidence
    if predicted_label == 0 and confidence > 0.7:
        print("✅ Model test passed!")
    else:
        print("⚠️ Model test results seem unusual, but proceeding...")

def predict_proba(self, texts):
    """Predict probabilities for LIME (batch processing)"""
    if isinstance(texts, str):
        texts = [texts]
    else:
        # normalize numpy/object arrays to a flat list of strings
        texts = [str(x) for x in np.array(texts, dtype=object).ravel().tolist()]

    premises, hyps = [], []
    for t in texts:
        if "</s>" in t:
            p, h = t.split("</s>", 1)
        else:
            # Fallback if LIME masks out the delimiter
            p, h = t, ""
        premises.append(p)
        hyps.append(h)

    inputs = self.tokenizer(
        premises,
        hyps,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=512
    )
    inputs = {k: v.to(self.device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = self.model(**inputs)
        probs = torch.softmax(outputs.logits, dim=1).detach().cpu().numpy()
    # Ensure 2D shape
    if probs.ndim == 1:
        probs = probs.reshape(1, -1)

```

```

    return probs

def predict_single(self, text):
    """Get single prediction with details"""
    probs = self.predict_proba([text])[0]
    predicted_label = int(np.argmax(probs))

    return {
        'probabilities': probs,
        'predicted_label': predicted_label,
        'predicted_class': self.label_mapping[predicted_label],
        'confidence': float(np.max(probs)),
        'all_probs': {self.label_mapping[i]: probs[i] for i in range(len(probs))}
    }

```

PART 3: LIME EXPLANATIONS WITH PROPER EVALUATION

```

In [ ]: def generate_lime_explanations(classifier, processed_snli, dirs, num_examples=50,
                                       lime_num_samples=1000, chunk_size=64,
                                       force_rebuild=False, save_every=10):

    """
    Generate LIME explanations with proper RoBERTa integration.
    Resumable: keeps existing items, appends new ones until num_examples.
    """

    lime_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "lime_explanations_roberta.json")

    # Load any existing data (resumable)
    existing = []
    if file_nonempty(lime_path) and not force_rebuild:
        try:
            raw = read_json(lime_path)
            existing = raw.get("data", raw) if isinstance(raw, dict) else raw
            print(f"▶ Found existing LIME explanations at {lime_path} with {len(existing)} items")
        except Exception as e:
            print(f"⚠ Failed to read existing LIME file ({e}). Will regenerate.")

    processed_ids = {int(r["id"]) for r in existing if "id" in r}
    target_total = int(num_examples)
    to_go = max(0, target_total - len(existing))
    if to_go == 0:
        print("▶ Already have requested number of LIME items; returning cached.")
        return existing[:target_total]

    print(f"🔍 Generating LIME explanations for {to_go} new examples (target={target_total}).")
    print(f"    LIME samples: {lime_num_samples}, Chunk size: {chunk_size}")

    # Inner batched predictor with adaptive chunking + OOM recovery
    def _lime_predict(batch, chunk_size=chunk_size):
        if isinstance(batch, str):
            texts = [batch]
        else:
            texts = [str(x) for x in np.array(batch, dtype=object).ravel().tolist()]

        rows = []
        i = 0
        while i < len(texts):
            sub = texts[i:i+chunk_size]

```



```

try:
    probs = classifier.predict_proba(sub)
except RuntimeError as e:
    if "CUDA out of memory" in str(e) and chunk_size > 8:
        torch.cuda.empty_cache()
        chunk_size = max(8, chunk_size // 2)
        print(f"⚠ OOM – reducing chunk_size to {chunk_size} and retrying...")
        continue
    raise
probs = np.asarray(probs)
if probs.ndim == 1: probs = probs.reshape(1, -1)
# defensive per-item fallback
if probs.shape[0] != len(sub):
    fixed = []
    for t in sub:
        p = classifier.predict_proba([t])
        p = np.asarray(p)
        if p.ndim == 1: p = p.reshape(1, -1)
        fixed.append(p[0])
    probs = np.vstack(fixed)
rows.append(probs)
i += len(sub)

out = np.vstack(rows)
assert out.shape[0] == len(texts), f"probs rows {out.shape[0]} != texts {len(texts)}"
return out

explainer = LimeTextExplainer(class_names=MNLI_LABELS, verbose=False, random_state=42)
new_items = []
seen = 0

for ex in tqdm(processed_snli, desc="LIME Explanations"):
    if len(existing) + len(new_items) >= target_total:
        break
    if int(ex["id"]) in processed_ids:
        continue

    try:
        text = ex["input_text"]
        t0 = time.perf_counter()
        prediction = classifier.predict_single(text)
        explanation = explainer.explain_instance(
            text,
            _lime_predict,
            num_features=10,
            num_samples=int(lime_num_samples),
        )
        latency = time.perf_counter() - t0
        item = {
            "id": int(ex["id"]),
            "premise": ex["premise"],
            "hypothesis": ex["hypothesis"],
            "input_text": text,
            "true_label": int(ex["label"]),
            "true_label_name": ex["label_name"],
            "predicted_label": int(prediction["predicted_label"]),
            "predicted_class": prediction["predicted_class"],
            "confidence": float(prediction["confidence"]),
            "all_probabilities": {k: float(v) for k, v in prediction["all_probs"].items()},
            "lime_attributions": [(w, float(s)) for (w, s) in explanation.as_list()],

```

```

        "lime_score": float(explanation.score),
        "lime_runtime_sec": float(latency),
    }
    new_items.append(item)
    seen += 1

    # checkpoint
    if seen % save_every == 0:
        merged = {"schema_version": SCHEMA_VERSION, "data": existing + new_items}
        write_json(lime_path, merged)
        print(f"📁 Checkpoint: saved {len(existing)+len(new_items)}/{target_total} it

    except Exception as e:
        print(f"❌ Error on id={ex.get('id')}: {e}")

merged = {"schema_version": SCHEMA_VERSION, "data": existing + new_items}
write_json(lime_path, merged)
print(f"✅ Generated {len(new_items)} new LIME explanations (total={len(existing)+len(new_items)})
print(f"✅ Saved to: {lime_path}")
return (existing + new_items)[:target_total]

```

```

In [ ]: def filter_stopwords_from_lime(lime_results, dirs):
    """Remove stopwords from LIME attributions for cleaner analysis"""
    if not lime_results:
        print("🔊 No LIME results; skipping stopwords filtering.")
        return []
    print("🔪 Filtering stopwords from LIME attributions...")

    # Setup stopwords
    stopwords_set = set(stopwords.words('english'))
    additional_stopwords = {"</s>", "[SEP]", "[CLS]", "[PAD]", "[UNK]", "[MASK]", "SEP"}
    stopwords_set.update(additional_stopwords)

    filtered_results = []
    for result in lime_results:
        # Filter attributions
        filtered_attributions = [
            (token, score) for token, score in result["lime_attributions"]
            if token.lower() not in stopwords_set and len(token.strip()) > 1
        ]

        # Create filtered result
        filtered_result = result.copy()
        filtered_result["lime_attributions_filtered"] = filtered_attributions
        filtered_result["original_attribution_count"] = len(result["lime_attributions"])
        filtered_result["filtered_attribution_count"] = len(filtered_attributions)

        filtered_results.append(filtered_result)

    # Save filtered results with schema
    filtered_data_with_schema = {
        "schema_version": SCHEMA_VERSION,
        "data": filtered_results
    }
    filtered_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "lime_explanations_filtered.json")
    write_json(filtered_path, filtered_data_with_schema)

    print(f"✅ Filtered results saved to: {filtered_path}")
    print(f"Average attribution reduction: {np.mean([r['original_attribution_count'] - r['fil

```

```
return filtered_results
```

PART 4: EVALUATION METRICS

```
In [10]: def compute_faithfulness(classifier, original_text, top_tokens):
        """Compute faithfulness by removing top-k tokens"""
        try:
            # Remove top-k tokens
            words = original_text.split()
            perturbed_words = [w for w in words if w not in top_tokens]
            perturbed_text = " ".join(perturbed_words)

            if not perturbed_text.strip():
                return 0.0

            # Get predictions
            original_prob = classifier.predict_proba([original_text])[0].max()
            perturbed_prob = classifier.predict_proba([perturbed_text])[0].max()

            return abs(original_prob - perturbed_prob)

        except Exception:
            return 0.0

def compute_sufficiency(classifier, original_text, top_tokens):
    """Compute sufficiency by keeping only top-k tokens"""
    try:
        words = original_text.split()
        kept_words = [w for w in words if w in top_tokens]
        sufficient_text = " ".join(kept_words)

        if not sufficient_text.strip():
            return 0.0

        return classifier.predict_proba([sufficient_text])[0].max()

    except Exception:
        return 0.0

def compute_comprehensiveness(classifier, original_text, top_tokens):
    """Compute comprehensiveness"""
    try:
        words = original_text.split()
        remaining_words = [w for w in words if w not in top_tokens]
        remaining_text = " ".join(remaining_words)

        if not remaining_text.strip():
            return 0.0

        original_prob = classifier.predict_proba([original_text])[0].max()
        remaining_prob = classifier.predict_proba([remaining_text])[0].max()

        return original_prob - remaining_prob

    except Exception:
        return 0.0
```

```

In [ ]: def compute_evaluation_metrics(classifier, lime_results, k=3, dirs=None, save_every=10):
        """Compute faithfulness, sufficiency, and comprehensiveness"""
        print(f"📊 Computing evaluation metrics (k={k})...")
        out_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "metrics_partial.json") if dirs else None

        done_ids = set()
        partial = []
        if out_path and file_nonempty(out_path):
            try:
                raw = read_json(out_path)
                partial = raw.get("data", raw) if isinstance(raw, dict) else raw
                done_ids = {int(r["id"]) for r in partial if "id" in r}
                print(f"🔄 Resuming metrics from {len(done_ids)} items.")
            except Exception:
                pass

        results = list(partial)
        since_last = 0

        for result in tqdm(lime_results, desc="Computing metrics"):
            try:
                text = result["input_text"]
                attributions = result.get("lime_attributions_filtered", result["lime_attributions"])

                if len(attributions) < k:
                    continue

                rid = int(result["id"])
                if rid in done_ids:
                    continue

                # Get top-k tokens
                sorted_attrs = sorted(attributions, key=lambda x: abs(x[1]), reverse=True)
                top_tokens = [token for token, _ in sorted_attrs[:k]]

                # Compute metrics
                faithfulness = compute_faithfulness(classifier, text, top_tokens)
                sufficiency = compute_sufficiency(classifier, text, top_tokens)
                comprehensiveness = compute_comprehensiveness(classifier, text, top_tokens)

                metric_result = {
                    "id": rid,
                    "faithfulness": faithfulness,
                    "sufficiency": sufficiency,
                    "comprehensiveness": comprehensiveness,
                    "top_tokens": top_tokens,
                    "k": k
                }

                results.append(metric_result)
                since_last += 1

                if out_path and since_last >= save_every:
                    write_json(out_path, with_schema({"data": results}))
                    since_last = 0

            except Exception as e:
                print(f"❌ Error computing metrics for example {result['id']}: {e}")
                continue

```

```

if out_path:
    write_json(out_path, with_schema({"data": results}))

return results

```

```

In [ ]: def perform_sanity_checks(classifier, lime_results, dirs):
        """Perform sanity checks on explanations"""
        print("🔍 Performing sanity checks...")

        # Separate correct and incorrect predictions
        correct_examples = [r for r in lime_results if r['true_label_name'] == r['predicted_class']]
        incorrect_examples = [r for r in lime_results if r['true_label_name'] != r['predicted_class']]

        print(f"Found {len(correct_examples)} correct, {len(incorrect_examples)} incorrect predictions")

        sanity_results = {
            "correct_examples": [],
            "incorrect_examples": [],
            "shuffle_test_results": []
        }

        # Check top 5 correct examples
        for i, example in enumerate(correct_examples[:5]):
            attributions = example.get("lime_attributions_filtered", example["lime_attributions"])
            top_5_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=True)[:5]

            sanity_results["correct_examples"].append({
                "id": example["id"],
                "predicted_class": example["predicted_class"],
                "confidence": example["confidence"],
                "top_tokens": [(token, float(score)) for token, score in top_5_tokens],
                "premise": example["premise"][:100] + "..." if len(example["premise"]) > 100 else example["premise"],
                "hypothesis": example["hypothesis"][:100] + "..." if len(example["hypothesis"]) > 100 else example["hypothesis"]
            })

        # Check top 5 incorrect examples
        for i, example in enumerate(incorrect_examples[:5]):
            attributions = example.get("lime_attributions_filtered", example["lime_attributions"])
            top_5_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=True)[:5]

            sanity_results["incorrect_examples"].append({
                "id": example["id"],
                "true_class": example["true_label_name"],
                "predicted_class": example["predicted_class"],
                "confidence": example["confidence"],
                "top_tokens": [(token, float(score)) for token, score in top_5_tokens],
                "premise": example["premise"][:100] + "..." if len(example["premise"]) > 100 else example["premise"],
                "hypothesis": example["hypothesis"][:100] + "..." if len(example["hypothesis"]) > 100 else example["hypothesis"]
            })

        # Shuffle test on 3 examples
        print("🔄 Running shuffle sanity test...")
        for i, example in enumerate(lime_results[:3]):
            original_text = example["input_text"]

            # Shuffle words
            words = original_text.split()
            random.shuffle(words)
            shuffled_text = " ".join(words)

```

```

# Compute faithfulness for both
attributions = example.get("lime_attributions_filtered", example["lime_attributions"])
if len(attributions) >= 3:
    top_tokens = [token for token, _ in sorted(attributions, key=lambda x: abs(x[1]),

original_faithfulness = compute_faithfulness(classifier, original_text, top_tokens)
shuffled_faithfulness = compute_faithfulness(classifier, shuffled_text, top_tokens)

sanity_results["shuffle_test_results"].append({
    "id": example["id"],
    "original_faithfulness": float(original_faithfulness),
    "shuffled_faithfulness": float(shuffled_faithfulness),
    "faithfulness_drop": float(original_faithfulness - shuffled_faithfulness)
})

# Save sanity check results
sanity_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "sanity_check_results.json")
write_json(sanity_path, sanity_results)
print(f"✅ Sanity check results saved to: {sanity_path}")

return sanity_results

```

```

In [ ]: def error_analysis(lime_results, dirs):
    """Analyze errors and create confusion matrix"""
    print("🔍 Performing error analysis...")

    # Get 10 most confident wrong predictions
    wrong_predictions = [r for r in lime_results if r['true_label'] != r['predicted_label']]
    most_confident_wrong = sorted(wrong_predictions, key=lambda x: x['confidence'], reverse=True)

    error_analysis_results = []
    for example in most_confident_wrong:
        attributions = example.get("lime_attributions_filtered", example["lime_attributions"])
        top_10_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=True)[:10]

        error_analysis_results.append({
            "id": example["id"],
            "premise": example["premise"],
            "hypothesis": example["hypothesis"],
            "true_label": example["true_label_name"],
            "predicted_label": example["predicted_class"],
            "confidence": example["confidence"],
            "top_10_attributions": [(token, float(score)) for token, score in top_10_tokens]
        })

    # Create confusion matrix
    y_true = [SNLI_TO_MNLI_IDX[int(r['true_label'])] for r in lime_results] # remap SNLI -> MNLI
    y_pred = [int(r['predicted_label']) for r in lime_results]

    cm = confusion_matrix(y_true, y_pred)
    label_names = ['entailment', 'neutral', 'contradiction']

    # Save error analysis
    error_data = {
        "schema_version": SCHEMA_VERSION,
        "most_confident_wrong_predictions": error_analysis_results,
        "confusion_matrix": cm.tolist(),
        "label_names": label_names,
        "summary": {

```

```

        "total_examples": len(lime_results),
        "wrong_predictions": len(wrong_predictions),
        "error_rate": len(wrong_predictions) / len(lime_results)
    }
}

error_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "error_analysis.json")
write_json(error_path, error_data)
print(f"✅ Error analysis saved to: {error_path}")

return error_data

```

```

In [ ]: def lime_stability_sweep(classifier, processed_snli, dirs):
    """Mini parameter sweep for LIME stability"""
    print("🔄 Running LIME stability mini-sweep...")

    stability_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "lime_stability_sweep.json")
    # Parameter grid
    param_grid = [
        {"num_samples": 500, "kernel_width": None},
        {"num_samples": 1000, "kernel_width": None},
        # {"num_samples": 500, "kernel_width": 25},
        # {"num_samples": 1000, "kernel_width": 25}
    ]

    # Test on 10 examples
    test_examples = processed_snli[:10] # keep small so it's fast/visible
    total_tasks = len(param_grid) * len(test_examples)

    # reuse the robust LIME callback you wrote earlier
    def _lime_predict(batch):
        if isinstance(batch, str):
            texts = [batch]
        else:
            texts = [str(x) for x in np.array(batch, dtype=object).ravel().tolist()]
        return classifier.predict_proba(texts)

    stability_results = []
    with tqdm(total=total_tasks, desc="Stability sweep", unit="ex", dynamic_ncols=True) as pbar:
        for pi, params in enumerate(param_grid, 1):
            kw = params["kernel_width"]
            kw_label = "default" if kw is None else kw

            # Build explainer (omit kernel_width when None)
            init_kwargs = {
                "class_names": list(classifier.label_mapping.values()),
                "verbose": False,
                "random_state": 42,
            }
            if kw is not None:
                init_kwargs["kernel_width"] = float(kw)

            explainer = LimeTextExplainer(**init_kwargs)

            metrics = []
            for ei, ex in enumerate(test_examples, 1):
                try:
                    text = ex["input_text"]
                    explanation = explainer.explain_instance(
                        text,

```

```

        _lime_predict,
        num_features=10,
        num_samples=int(params["num_samples"]),
    )
    attributions = explanation.as_list()
    if len(attributions) >= 3:
        top_tokens = [t for t, _ in sorted(attributions, key=lambda x: abs(x[0]))]
        f = compute_faithfulness(classifier, text, top_tokens)
        s = compute_sufficiency(classifier, text, top_tokens)
        metrics.append({"faithfulness": float(f), "sufficiency": float(s)})

    except Exception as e:
        tqdm.write(f"Error (ns={params['num_samples']}, kw={kw_label}, ex={e}):")

    # update the global bar + postfix info
    pbar.set_postfix_str(f"ns={params['num_samples']}, kw={kw_label}, ex {e}/{len(params)}")
    pbar.update(1)

    if metrics:
        stability_results.append({
            "params": params,
            "avg_faithfulness": float(np.mean([m["faithfulness"] for m in metrics])),
            "avg_sufficiency": float(np.mean([m["sufficiency"] for m in metrics])),
            "num_examples": len(metrics),
        })
    write_json(stability_path, with_schema({"results": stability_results}))

# Save stability results
stability_data = {
    "schema_version": SCHEMA_VERSION,
    "results": stability_results
}
write_json(stability_path, stability_data)
print(f"✅ Stability sweep results saved to: {stability_path}")

return stability_results

```

```

In [ ]: def create_comprehensive_plots(lime_results, metrics_results, dirs):
    """Create comprehensive visualization plots"""
    print(f"📊 Creating comprehensive visualization plots...")

    plots_dir = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots")
    ensure_dir(plots_dir)

    # Set style
    plt.style.use('default')
    sns.set_palette("husl")

    # 1. Model Performance Analysis
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # Confidence distribution
    confidences = [r['confidence'] for r in lime_results]
    axes[0, 0].hist(confidences, bins=20, alpha=0.7, color='skyblue', edgecolor='black')
    axes[0, 0].set_title('Model Confidence Distribution')
    axes[0, 0].set_xlabel('Confidence Score')
    axes[0, 0].set_ylabel('Frequency')
    axes[0, 0].axvline(np.mean(confidences), color='red', linestyle='--',
                      label=f'Mean: {np.mean(confidences):.3f}')
    axes[0, 0].legend()

```



```

# Label distribution
true_labels = [r['true_label_name'] for r in lime_results]
pred_labels = [r['predicted_class'] for r in lime_results]

label_counts = Counter(true_labels)
axes[0, 1].bar(label_counts.keys(), label_counts.values(), alpha=0.7, color='lightgreen')
axes[0, 1].set_title('True Label Distribution')
axes[0, 1].set_xlabel('Labels')
axes[0, 1].set_ylabel('Count')
axes[0, 1].tick_params(axis='x', rotation=45)

# Accuracy by confidence bins
df_results = pd.DataFrame(lime_results)
df_results['correct'] = df_results['true_label_name'] == df_results['predicted_class']
df_results['conf_bin'] = pd.cut(df_results['confidence'], bins=5,
                                labels=['Very Low', 'Low', 'Medium', 'High', 'Very High'])
conf_acc = df_results.groupby('conf_bin')['correct'].mean()

axes[1, 0].bar(range(len(conf_acc)), conf_acc.values, alpha=0.7, color='orange')
axes[1, 0].set_title('Accuracy by Confidence Level')
axes[1, 0].set_xlabel('Confidence Bins')
axes[1, 0].set_ylabel('Accuracy')
axes[1, 0].set_xticks(range(len(conf_acc)))
axes[1, 0].set_xticklabels(conf_acc.index, rotation=45)

# Confusion matrix
true_labels_numeric = [r['true_label_name'] for r in lime_results]
pred_labels_numeric = [r['predicted_class'] for r in lime_results]
cm = confusion_matrix(true_labels_numeric, pred_labels_numeric)

label_names = ['entailment', 'neutral', 'contradiction']
im = axes[1, 1].imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
axes[1, 1].set_title('Confusion Matrix')
tick_marks = np.arange(len(label_names))
axes[1, 1].set_xticks(tick_marks)
axes[1, 1].set_yticks(tick_marks)
axes[1, 1].set_xticklabels(label_names, rotation=45)
axes[1, 1].set_yticklabels(label_names)

# Add text annotations
thresh = cm.max() / 2.
for i, j in np.ndindex(cm.shape):
    axes[1, 1].text(j, i, format(cm[i, j], 'd'),
                    ha="center", va="center",
                    color="white" if cm[i, j] > thresh else "black")

plt.tight_layout()
plt.savefig(os.path.join(plots_dir, "model_performance_analysis.png"), dpi=300, bbox_inches='tight')
plt.close()

# 2. LIME Attribution Analysis
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# Attribution score distribution
all_scores = []
for result in lime_results:
    attributions = result.get('lime_attributions_filtered', result['lime_attributions'])
    scores = [abs(score) for _, score in attributions]
    all_scores.extend(scores)

```

```

axes[0, 0].hist(all_scores, bins=30, alpha=0.7, color='purple', edgecolor='black')
axes[0, 0].set_title('LIME Attribution Score Distribution')
axes[0, 0].set_xlabel('Absolute Attribution Score')
axes[0, 0].set_ylabel('Frequency')
axes[0, 0].axvline(np.mean(all_scores), color='red', linestyle='--',
                  label=f'Mean: {np.mean(all_scores):.3f}')
axes[0, 0].legend()

# LIME score distribution
lime_scores = [r['lime_score'] for r in lime_results]
axes[0, 1].hist(lime_scores, bins=20, alpha=0.7, color='coral', edgecolor='black')
axes[0, 1].set_title('LIME Score Distribution')
axes[0, 1].set_xlabel('LIME Score')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].axvline(np.mean(lime_scores), color='red', linestyle='--',
                  label=f'Mean: {np.mean(lime_scores):.3f}')
axes[0, 1].legend()

# Confidence vs Faithfulness scatter (if metrics available)
if metrics_results:
    # Create mapping from id to metrics
    metrics_dict = {m['id']: m for m in metrics_results}

    plot_data = []
    for result in lime_results:
        if result['id'] in metrics_dict:
            plot_data.append({
                'confidence': result['confidence'],
                'faithfulness': metrics_dict[result['id']]['faithfulness']
            })

    if plot_data:
        conf_vals = [d['confidence'] for d in plot_data]
        faith_vals = [d['faithfulness'] for d in plot_data]

        axes[1, 0].scatter(conf_vals, faith_vals, alpha=0.6, color='green')
        axes[1, 0].set_title('Confidence vs Faithfulness')
        axes[1, 0].set_xlabel('Model Confidence')
        axes[1, 0].set_ylabel('Faithfulness Score')

        # Add correlation
        correlation = np.corrcoef(conf_vals, faith_vals)[0, 1]
        axes[1, 0].text(0.05, 0.95, f'Correlation: {correlation:.3f}',
                      transform=axes[1, 0].transAxes, fontsize=12,
                      verticalalignment='top', bbox=dict(boxstyle='round', facecolor='white'))

# Top contributing words
word_scores = defaultdict(list)
for result in lime_results:
    attributions = result.get('lime_attributions_filtered', result['lime_attributions'])
    for word, score in attributions:
        word_scores[word.lower()].append(abs(score))

# Get top 15 words by average absolute score
avg_word_scores = {word: np.mean(scores) for word, scores in word_scores.items()}
top_words = sorted(avg_word_scores.items(), key=lambda x: x[1], reverse=True)[:15]

words, scores = zip(*top_words)
axes[1, 1].barh(range(len(words)), scores, alpha=0.7, color='teal')

```

```

axes[1, 1].set_yticks(range(len(words)))
axes[1, 1].set_yticklabels(words)
axes[1, 1].set_title('Top 15 Contributing Words')
axes[1, 1].set_xlabel('Average Absolute Attribution Score')

plt.tight_layout()
plt.savefig(os.path.join(plots_dir, "lime_attribution_analysis.png"), dpi=300, bbox_inches='tight')
plt.close()

# 3. Metrics Analysis (if available)
if metrics_results:
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    metrics_df = pd.DataFrame(metrics_results)

    # Metrics comparison
    metric_means = [metrics_df['faithfulness'].mean(),
                    metrics_df['sufficiency'].mean(),
                    metrics_df['comprehensiveness'].mean()]
    metric_names = ['Faithfulness', 'Sufficiency', 'Comprehensiveness']

    bars = axes[0].bar(metric_names, metric_means, alpha=0.7,
                      color=['red', 'blue', 'green'])
    axes[0].set_title('Average Attribution Metrics')
    axes[0].set_ylabel('Score')
    axes[0].tick_params(axis='x', rotation=45)

    # Add value labels on bars
    for bar, value in zip(bars, metric_means):
        axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                     f'{value:.3f}', ha='center', va='bottom')

    # Metrics correlation
    correlation_matrix = metrics_df[['faithfulness', 'sufficiency', 'comprehensiveness']]

    im = axes[1].imshow(correlation_matrix, cmap='RdYlBu', vmin=-1, vmax=1)
    axes[1].set_title('Metrics Correlation Matrix')
    axes[1].set_xticks(range(len(correlation_matrix.columns)))
    axes[1].set_yticks(range(len(correlation_matrix.columns)))
    axes[1].set_xticklabels(correlation_matrix.columns, rotation=45)
    axes[1].set_yticklabels(correlation_matrix.columns)

    # Add correlation values
    for i in range(len(correlation_matrix.columns)):
        for j in range(len(correlation_matrix.columns)):
            axes[1].text(j, i, f'{correlation_matrix.iloc[i, j]:.2f}',
                         ha='center', va='center', color='white', fontweight='bold')

    plt.colorbar(im, ax=axes[1])
    plt.tight_layout()
    plt.savefig(os.path.join(plots_dir, "metrics_analysis.png"), dpi=300, bbox_inches='tight')
    plt.close()

print(f"✅ All plots saved to: {plots_dir}")

```

```

In [ ]: def bootstrap_confidence_intervals(metrics_results, n_bootstrap=1000):
        """Compute bootstrap confidence intervals for metrics"""
        if not metrics_results:
            return {}

```

```

print("🔍 Computing bootstrap confidence intervals...")

metrics_df = pd.DataFrame(metrics_results)
bootstrap_results = {}

for metric in ['faithfulness', 'sufficiency', 'comprehensiveness']:
    if metric in metrics_df.columns:
        values = metrics_df[metric].values
        bootstrap_means = []

        for _ in range(n_bootstrap):
            # Resample with replacement
            resampled = np.random.choice(values, size=len(values), replace=True)
            bootstrap_means.append(np.mean(resampled))

        # Calculate 95% CI
        ci_lower = np.percentile(bootstrap_means, 2.5)
        ci_upper = np.percentile(bootstrap_means, 97.5)

        bootstrap_results[metric] = {
            "mean": float(np.mean(values)),
            "ci_lower": float(ci_lower),
            "ci_upper": float(ci_upper),
            "std": float(np.std(bootstrap_means))
        }

print("✅ Bootstrap confidence intervals computed")
return bootstrap_results

def generate_report(lime_results, metrics_results, dirs, config):
    """Generate comprehensive markdown report"""
    print("📄 Generating SNLI LIME report...")

    # Calculate basic statistics
    total_examples = len(lime_results)
    correct_predictions = sum(1 for r in lime_results if r['true_label_name'] == r['predicted_class'])
    accuracy = correct_predictions / total_examples
    avg_confidence = np.mean([r['confidence'] for r in lime_results])

    # Metrics statistics
    metrics_stats = {}
    if metrics_results:
        metrics_df = pd.DataFrame(metrics_results)
        for metric in ['faithfulness', 'sufficiency', 'comprehensiveness']:
            if metric in metrics_df.columns:
                metrics_stats[metric] = {
                    'mean': metrics_df[metric].mean(),
                    'std': metrics_df[metric].std(),
                    'min': metrics_df[metric].min(),
                    'max': metrics_df[metric].max()
                }

    # Bootstrap CIs
    bootstrap_cis = bootstrap_confidence_intervals(metrics_results)

    # Select qualitative examples
    # Top 3 correct examples by confidence
    correct_examples = [r for r in lime_results if r['true_label_name'] == r['predicted_class']]
    top_correct = sorted(correct_examples, key=lambda x: x['confidence'], reverse=True)[:3]

```

```

# Top 2 incorrect examples by confidence
incorrect_examples = [r for r in lime_results if r['true_label_name'] != r['predicted_label_name']]
top_incorrect = sorted(incorrect_examples, key=lambda x: x['confidence'], reverse=True)[:2]

# Generate report content
report_content = f"""# SNLI LIME Explanation Analysis Report

## Executive Summary

**Dataset**: SNLI (Stanford Natural Language Inference)
**Model**: {config['model_name']}
**Analysis Date**: {config['timestamp'][:10]}
**Total Examples**: {total_examples}
**Git Hash**: {config['git_hash']}

### Key Results
- **Model Accuracy**: {accuracy:.3f} ({correct_predictions}/{total_examples})
- **Average Confidence**: {avg_confidence:.3f}
- **LIME Parameters**: {config['lime_num_samples']} samples, {config['lime_num_features']} features
- **Random Seed**: {config['random_seed']}

## Attribution Metrics Results

"""

if metrics_stats:
    report_content += "### Faithfulness, Sufficiency, and Comprehensiveness\n\n"
    report_content += "| Metric | Mean | Std | Min | Max | 95% CI |\n"
    report_content += "|-----|-----|-----|-----|-----|-----|\n"

    for metric, stats in metrics_stats.items():
        ci_info = ""
        if metric in bootstrap_cis:
            ci = bootstrap_cis[metric]
            ci_info = f"[{ci['ci_lower']:.3f}, {ci['ci_upper']:.3f}]"

        report_content += f"| {metric.title()} | {stats['mean']:.3f} | {stats['std']:.3f} | {ci_info} |\n"

report_content += "\n## LIME Configuration\n\n"
report_content += f"- **Number of Samples**: {config['lime_num_samples']}\n"
report_content += f"- **Number of Features**: {config['lime_num_features']}\n"
report_content += f"- **Top-k Tokens for Metrics**: {config['k_tokens']}\n"
report_content += f"- **Chunk Size**: {config['chunk_size']}\n"

report_content += "\n## Qualitative Examples\n\n"
report_content += "### Top Correct Predictions\n\n"

for i, example in enumerate(top_correct):
    attributions = example.get('lime_attributions_filtered', example['lime_attributions'])
    top_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=True)[:5]

    report_content += f"**Example {i+1}** (Confidence: {example['confidence']:.3f})\n"
    report_content += f"- **Premise**: {example['premise']}\n"
    report_content += f"- **Hypothesis**: {example['hypothesis']}\n"
    report_content += f"- **Predicted**: {example['predicted_class']}\n"
    report_content += f"- **Top Attributions**: "

    token_strs = [f"{token}({score:.3f})" for token, score in top_tokens]
    report_content += ", ".join(token_strs) + "\n\n"

```

```

if top_incorrect:
    report_content += "### Top Incorrect Predictions\n\n"

    for i, example in enumerate(top_incorrect):
        attributions = example.get('lime_attributions_filtered', example['lime_attribution'])
        top_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=True)[:5]

        report_content += f"***Example {i+1}** (Confidence: {example['confidence']:.3f})\n"
        report_content += f"- **Premise**: {example['premise']}\n"
        report_content += f"- **Hypothesis**: {example['hypothesis']}\n"
        report_content += f"- **True**: {example['true_label_name']}, **Predicted**: {example['predicted_label']}\n"
        report_content += f"- **Top Attributions**: "

        token_strs = [f"{token}({score:.3f})" for token, score in top_tokens]
        report_content += ", ".join(token_strs) + "\n\n"

report_content += "\n## Known Caveats\n\n"
report_content += "- **Delimiter Handling**: Uses `</s>` separator between premise and hypothesis\n"
report_content += "- **Stopword Filtering**: Removes common English stopwords and special characters\n"
report_content += "- **Limited Sample Size**: Analysis based on 50 examples for speed\n"
report_content += "- **Perturbation Sensitivity**: LIME results may vary with different noise levels\n"

report_content += "\n## Next Steps for GRACE Framework\n\n"
report_content += "1. **Dynamic Retrieval-Guided Attribution (DRGA)**: Implement confidence-aware retrieval\n"
report_content += "2. **Explanation Hallucination Detection**: Build 4-type taxonomy detector\n"
report_content += "3. **Fairness Auditor**: Add demographic bias detection\n"
report_content += "4. **Unified Trust Score**: Combine faithfulness + factuality + fairness\n"

# Save report
report_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "SNLI_LIME_report.md")
with open(report_path, "w", encoding="utf-8") as f:
    f.write(report_content)

print(f"✅ Report saved to: {report_path}")
return report_content

def analyze_results_enhanced(lime_results, metrics_results, dirs):
    """Enhanced results analysis with comprehensive statistics"""
    print("📊 Enhanced results analysis...")

    if not lime_results:
        print("❌ No results; skipping analysis.")
        return {}

    # Basic performance metrics
    total = len(lime_results)
    correct = sum(1 for r in lime_results if r.get("true_label") == r.get("predicted_label"))
    accuracy = float(correct / total) if total else 0.0
    avg_confidence = float(np.mean([r.get("confidence", 0) for r in lime_results]))

    # Metrics analysis
    metrics_summary = {}
    if metrics_results:
        metrics_df = pd.DataFrame(metrics_results)
        metrics_summary = {
            "avg_faithfulness": float(metrics_df['faithfulness'].mean()),
            "avg_sufficiency": float(metrics_df['sufficiency'].mean()),
            "avg_comprehensiveness": float(metrics_df['comprehensiveness'].mean()),
            "std_faithfulness": float(metrics_df['faithfulness'].std()),
            "std_sufficiency": float(metrics_df['sufficiency'].std()),
        }

```

```

        "std_comprehensiveness": float(metrics_df['comprehensiveness'].std()),
        "num_examples": len(metrics_results)
    }

# Confidence distribution analysis
confidences = [r['confidence'] for r in lime_results]
confidence_analysis = {
    "high_confidence": sum(1 for c in confidences if c > 0.8),
    "medium_confidence": sum(1 for c in confidences if 0.5 < c <= 0.8),
    "low_confidence": sum(1 for c in confidences if c <= 0.5),
    "avg_confidence": float(np.mean(confidences)),
    "std_confidence": float(np.std(confidences))
}

# Build final results
final_results = {
    "schema_version": SCHEMA_VERSION,
    "evaluation_metrics": metrics_summary,
    "model_performance": {
        "accuracy": accuracy,
        "avg_confidence": avg_confidence,
        "correct_predictions": int(correct),
        "total_examples": int(total),
        "error_rate": float(1 - accuracy)
    },
    "confidence_analysis": confidence_analysis,
    "lime_explanations": lime_results,
    "individual_metrics": metrics_results if metrics_results else []
}

# Save results
results_path = os.path.join(dirs["LIME_OUTPUT_DIR"], "complete_evaluation_results.json")
write_json(results_path, final_results)

print(f"\n🔴 FINAL EVALUATION RESULTS:")
print(f"    Model Accuracy: {accuracy:.3f}")
print(f"    Average Confidence: {avg_confidence:.3f}")

if metrics_summary:
    print(f"    Average Faithfulness: {metrics_summary['avg_faithfulness']:.3f} ± {metrics_summary['std_faithfulness']:.3f}")
    print(f"    Average Sufficiency: {metrics_summary['avg_sufficiency']:.3f} ± {metrics_summary['std_sufficiency']:.3f}")
    print(f"    Average Comprehensiveness: {metrics_summary['avg_comprehensiveness']:.3f} ± {metrics_summary['std_comprehensiveness']:.3f}")

print(f"✅ Complete results saved to: {results_path}")
return final_results

def plot_runtime_distribution(lime_results, dirs):
    print("📊 Plotting LIME runtime distribution...")
    times = [r.get("lime_runtime_sec", np.nan) for r in lime_results]
    times = [t for t in times if not np.isnan(t)]
    if not times: return
    plt.figure(figsize=(8,5))
    plt.hist(times, bins=20, edgecolor="black", alpha=0.7)
    plt.title("Per-example LIME runtime")
    plt.xlabel("Seconds"); plt.ylabel("Count")
    outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots", "lime_runtime_hist.png")
    ensure_dir(os.path.dirname(outp))
    plt.tight_layout(); plt.savefig(outp, dpi=300); plt.close()

def plot_per_class_top_words(lime_results, dirs, by="predicted_class", top_n=15):

```



```

print("📊 Plotting top words per class...")
groups = {"entailment": [], "neutral": [], "contradiction": []}
for r in lime_results:
    lbl = r.get(by)
    if lbl not in groups: continue
    atts = r.get("lime_attributions_filtered", r.get("lime_attributions", []))
    groups[lbl].extend([ (w.lower(), abs(s)) for w,s in atts ])

for lbl, pairs in groups.items():
    if not pairs: continue
    from collections import defaultdict
    agg = defaultdict(list)
    for w,s in pairs: agg[w].append(s)
    avg = sorted([(w, float(np.mean(v))) for w,v in agg.items()],
                  key=lambda x: x[1], reverse=True)[:top_n])
    words, scores = zip(*avg)
    plt.figure(figsize=(8,6))
    plt.barh(range(len(words)), scores)
    plt.yticks(range(len(words)), words)
    plt.gca().invert_yaxis()
    plt.title(f"Top {top_n} contributing words - {lbl}")
    plt.xlabel("Avg |attribution|")
    outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots", f"top_words_{lbl}.png")
    ensure_dir(os.path.dirname(outp))
    plt.tight_layout(); plt.savefig(outp, dpi=300); plt.close()

def plot_reliability_diagram(lime_results, dirs, bins=10):
    print("📊 Plotting reliability diagram...")
    # max prob & correctness
    preds = [r["all_probabilities"].get(r["predicted_class"], r["confidence"]) for r in lime_results]
    correct = [int(r["true_label_name"] == r["predicted_class"]) for r in lime_results]
    preds = np.array(preds, dtype=float)
    correct = np.array(correct, dtype=float)

    # bin by confidence
    edges = np.linspace(0, 1, bins+1)
    idx = np.digitize(preds, edges) - 1
    acc, conf, cnt = [], [], []
    for b in range(bins):
        mask = idx == b
        if mask.any():
            acc.append(float(correct[mask].mean()))
            conf.append(float(preds[mask].mean()))
            cnt.append(int(mask.sum()))
        else:
            acc.append(np.nan); conf.append((edges[b]+edges[b+1])/2); cnt.append(0)

    plt.figure(figsize=(6,6))
    plt.plot([0,1],[0,1], '--', label='perfect')
    plt.plot(conf, acc, marker='o', label='model')
    plt.xlabel('Mean predicted confidence'); plt.ylabel('Empirical accuracy')
    plt.title('Reliability diagram'); plt.legend()
    outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots", "reliability.png")
    ensure_dir(os.path.dirname(outp))
    plt.tight_layout(); plt.savefig(outp, dpi=300); plt.close()

```

```

In [17]: import os
import numpy as np

def analyze_results(filtered_results, metrics_results, dirs):

```



```

"""Save a summary, accepting metrics_results as dict OR list OR (dict, list)."""
if not filtered_results:
    print("❗ No results; skipping analysis.")
    return {}

def safe_float(x):
    try:
        return float(x)
    except Exception:
        return np.nan

# 1) Normalize metrics_results into summary numbers
avg_faithfulness = np.nan
avg_sufficiency = np.nan
avg_comprehensiveness = np.nan
n = None
per_example = None

if isinstance(metrics_results, dict):
    # expected keys already there
    avg_faithfulness = safe_float(metrics_results.get("avg_faithfulness"))
    avg_sufficiency = safe_float(metrics_results.get("avg_sufficiency"))
    avg_comprehensiveness = safe_float(metrics_results.get("avg_comprehensiveness"))
    n = int(metrics_results.get("n", len(filtered_results)))
    per_example = metrics_results.get("per_example")

elif isinstance(metrics_results, (list, tuple)):
    # tuple like (summary_dict, per_list)?
    if isinstance(metrics_results, tuple) and len(metrics_results) == 2 and isinstance(metrics_results[0], dict):
        summ, per = metrics_results
        avg_faithfulness = safe_float(summ.get("avg_faithfulness"))
        avg_sufficiency = safe_float(summ.get("avg_sufficiency"))
        avg_comprehensiveness = safe_float(summ.get("avg_comprehensiveness"))
        n = int(summ.get("n", len(filtered_results)))
        per_example = per
    else:
        # treat as list of per-example dicts
        per_example = list(metrics_results)
        n = len(per_example)
        if n:
            avg_faithfulness = float(np.nanmean([safe_float(x.get("faithfulness")) for x in per_example]))
            avg_sufficiency = float(np.nanmean([safe_float(x.get("sufficiency")) for x in per_example]))
            avg_comprehensiveness = float(np.nanmean([safe_float(x.get("comprehensiveness")) for x in per_example]))
        else:
            avg_faithfulness = avg_sufficiency = avg_comprehensiveness = float("nan")
    else:
        # unknown type; fall back to computing only from filtered_results
        n = len(filtered_results)

# 2) Model performance from filtered_results (always available)
total = len(filtered_results)
correct = sum(1 for r in filtered_results if r.get("true_label_name") == r.get("predicted_label_name"))
avg_confidence = float(np.mean([safe_float(r.get("confidence")) for r in filtered_results]))
accuracy = float(correct / total) if total else 0.0

# 3) Build final summary (cast to native Python types)
final_results = {
    "evaluation": {
        "average_faithfulness": float(avg_faithfulness) if not np.isnan(avg_faithfulness) else 0.0,
        "average_sufficiency": float(avg_sufficiency) if not np.isnan(avg_sufficiency) else 0.0,
        "average_comprehensiveness": float(avg_comprehensiveness) if not np.isnan(avg_comprehensiveness) else 0.0,
        "n": n,
        "per_example": per_example
    }
}

```

```

        "average_comprehensiveness": float(avg_comprehensiveness) if not np.isnan(avg_comprehensiveness) else 0,
        "num_examples": int(n if n is not None else total),
    },
    "model_performance": {
        "accuracy": float(accuracy),
        "avg_confidence": float(avg_confidence),
        "correct": int(correct),
        "total": int(total),
    },
}

# (Optional) include per-example if you want it in the JSON
# if per_example is not None:
#     final_results["per_example"] = [
#         {
#             "id": int(r.get("id", i)),
#             "faithfulness": safe_float(r.get("faithfulness")),
#             "sufficiency": safe_float(r.get("sufficiency")),
#             "comprehensiveness": safe_float(r.get("comprehensiveness")),
#         } for i, r in enumerate(per_example)
#     ]

out_path = os.path.join(dirs["LIME_OUTPUT_DIR"], "complete_evaluation_results.json")
write_json(out_path, final_results) # uses your NumPy-safe writer
print(f"✅ Complete results saved to: {out_path}")
return final_results

```

```

In [ ]: def export_summary_csv(lime_results, metrics_results, dirs):
import pandas as pd
m = {r["id"]: r for r in metrics_results} if metrics_results else {}
rows = []
for r in lime_results:
    mid = r["id"]
    rows.append({
        "id": mid,
        "true": r["true_label_name"],
        "pred": r["predicted_class"],
        "conf": r["confidence"],
        "lime_score": r.get("lime_score", np.nan),
        "lime_runtime_sec": r.get("lime_runtime_sec", np.nan),
        "faithfulness": m.get(mid, {}).get("faithfulness", np.nan),
        "sufficiency": m.get(mid, {}).get("sufficiency", np.nan),
        "comprehensiveness": m.get(mid, {}).get("comprehensiveness", np.nan),
    })
df = pd.DataFrame(rows)
outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "summary.csv")
df.to_csv(outp, index=False, encoding="utf-8")
print(f"📄 Exported CSV: {outp}")

```

MAIN EXECUTION PIPELINE

```

In [ ]: def main(num_examples=50, lime_num_samples=1000, chunk_size=64, force_rebuild=False):
    """Enhanced main pipeline with all requested features"""
    print("🌀 Starting Enhanced ROBERTa-MNLI + LIME Pipeline for SNLI")
    print("🔧 New features: reproducibility, sanity checks, error analysis, stability sweep,

    # Set reproducible seeds
    set_all_seeds(42)

```

```

# Clear GPU memory
if torch.cuda.is_available():
    print("🔒 Clearing GPU memory...")
    torch.cuda.empty_cache()
    torch.cuda.ipc_collect()
    print("✅ GPU memory cleared.")
else:
    print("❗ No GPU available; skipping memory cleanup.")

try:
    # Step 1: Setup
    dirs = setup_directories()
    print("✅ Directories setup complete")

    # Save run configuration
    config = save_run_config(dirs, num_samples=num_examples,
                             lime_num_samples=lime_num_samples, chunk_size=chunk_size)

    # Step 2: Load and sample data
    snli_data = load_snli_dataset_fixed(dirs)
    snli_sample = sample_snli_dataset_fixed(snli_data, dirs, num_samples=num_examples)
    processed_snli = preprocess_snli_for_roberta(snli_sample, dirs)

    # Step 3: Load RoBERTa model with unit tests
    classifier = RoBERTaMNLIClassifier("roberta-large-mnli", use_fp16=True)

    # Step 4: Generate LIME explanations
    lime_results = generate_lime_explanations(classifier, processed_snli, dirs,
                                              num_examples=num_examples,
                                              lime_num_samples=lime_num_samples,
                                              chunk_size=chunk_size,
                                              force_rebuild=force_rebuild)

    # Step 5: Filter stopwords
    filtered_results = filter_stopwords_from_lime(lime_results, dirs)

    # Step 6: Evaluate metrics
    metrics_results = compute_evaluation_metrics(classifier, filtered_results, k=3, dirs=dirs)

    export_summary_csv(filtered_results, metrics_results, dirs)

    # Step 7: Sanity checks
    sanity_results = perform_sanity_checks(classifier, filtered_results, dirs)

    # Step 8: Error analysis
    error_results = error_analysis(filtered_results, dirs)

    # Step 9: LIME stability sweep
    stability_results = lime_stability_sweep(classifier, processed_snli, dirs)

    # Step 10: Enhanced analysis with plots
    final_results = analyze_results_enhanced(filtered_results, metrics_results, dirs)

    # Step 11: Create comprehensive visualizations
    create_comprehensive_plots(filtered_results, metrics_results, dirs)

    plot_runtime_distribution(filtered_results, dirs)
    plot_per_class_top_words(filtered_results, dirs, by="predicted_class", top_n=15)
    plot_reliability_diagram(filtered_results, dirs)

```

```

# Step 12: Generate report
report_content = generate_report(filtered_results, metrics_results, dirs, config)

print("\n🎉 Enhanced pipeline completed successfully!")
print("📁 Generated files:")
print(f"📁 Results: {dirs['LIME_OUTPUT_DIR']}/complete_evaluation_results.json")
print(f"🔧 Config: {dirs['LIME_OUTPUT_DIR']}/run_config.json")
print(f"🔍 Sanity: {dirs['LIME_OUTPUT_DIR']}/sanity_check_results.json")
print(f"❌ Errors: {dirs['LIME_OUTPUT_DIR']}/error_analysis.json")
print(f"📊 Stability: {dirs['LIME_OUTPUT_DIR']}/lime_stability_sweep.json")
print(f"📄 Report: {dirs['LIME_OUTPUT_DIR']}/SNLI_LIME_report.md")
print(f"📈 Plots: {dirs['LIME_OUTPUT_DIR']}/plots/")

return final_results

except Exception as e:
    print(f"❌ Enhanced pipeline failed: {e}")
    import traceback
    traceback.print_exc()
    return None

def main_with_args():
    """Main function with command line arguments"""
    parser = argparse.ArgumentParser(description='Enhanced GRACE LIME Pipeline')
    parser.add_argument('--num-samples', type=int, default=50, help='Number of examples to analyze')
    parser.add_argument('--lime-samples', type=int, default=1000, help='LIME perturbation samples')
    parser.add_argument('--chunk-size', type=int, default=64, help='Batch processing chunk size')
    parser.add_argument('--force', action='store_true', help='Force rebuild all cached files')

    args = parser.parse_args()

    return main(
        num_examples=args.num_samples,
        lime_num_samples=args.lime_samples,
        chunk_size=args.chunk_size,
        force_rebuild=args.force
    )

if __name__ == "__main__":
    # Run with default parameters (can be called with arguments)
    results = main(num_examples=1000, lime_num_samples=700, chunk_size=64, force_rebuild=False)

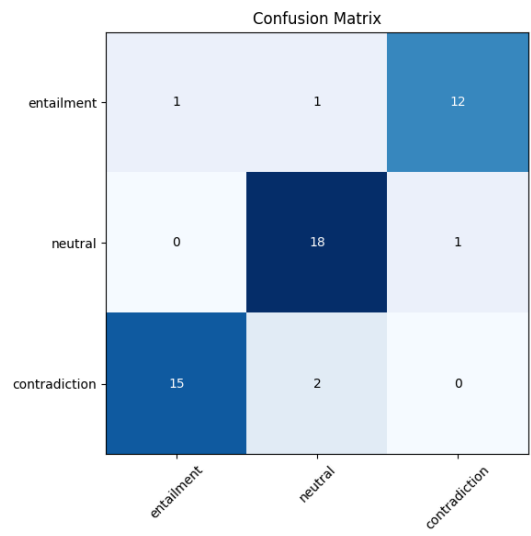
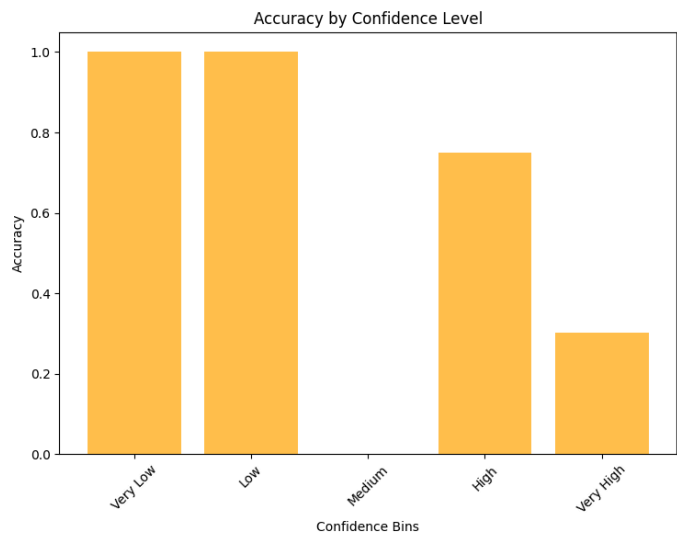
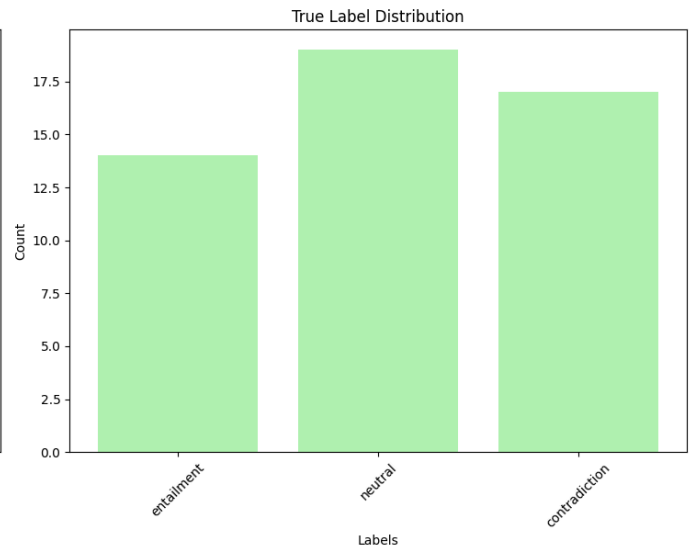
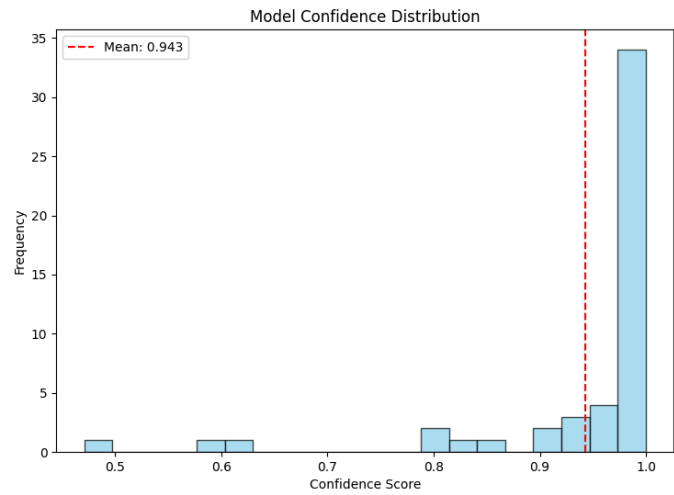
    # Alternative: Uncomment to use command line arguments
    # results = main_with_args()

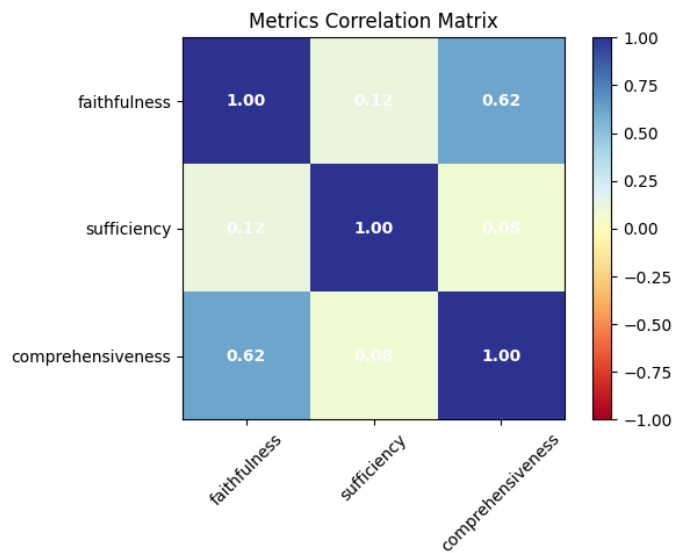
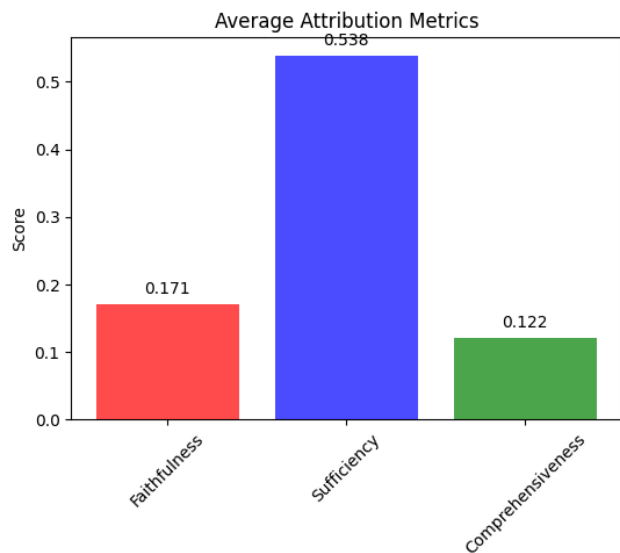
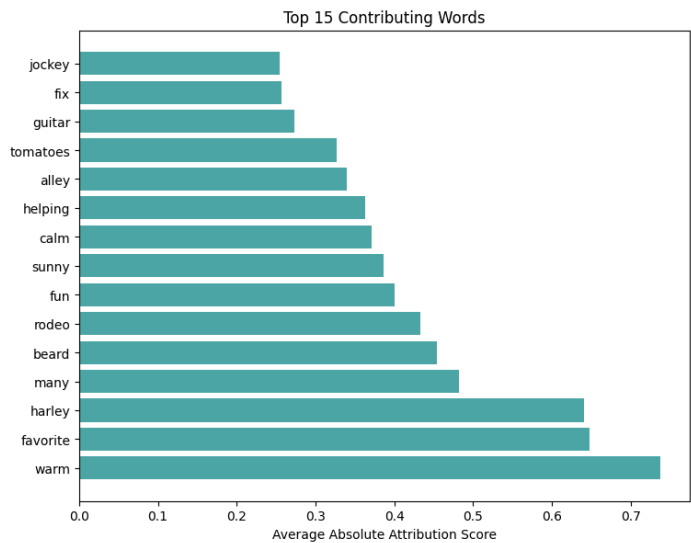
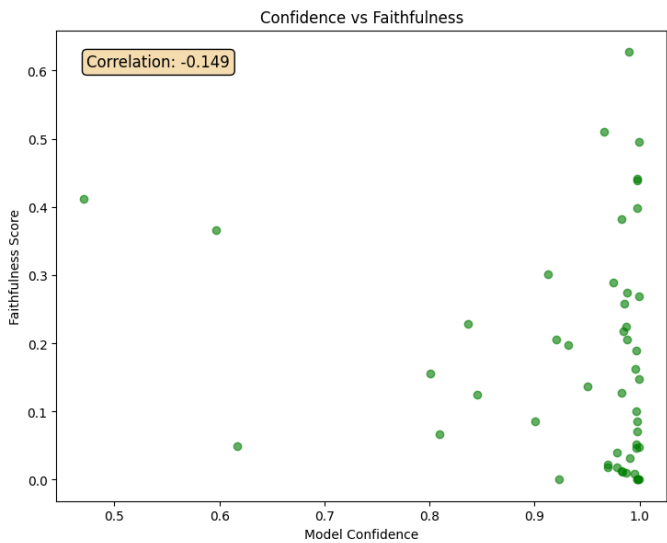
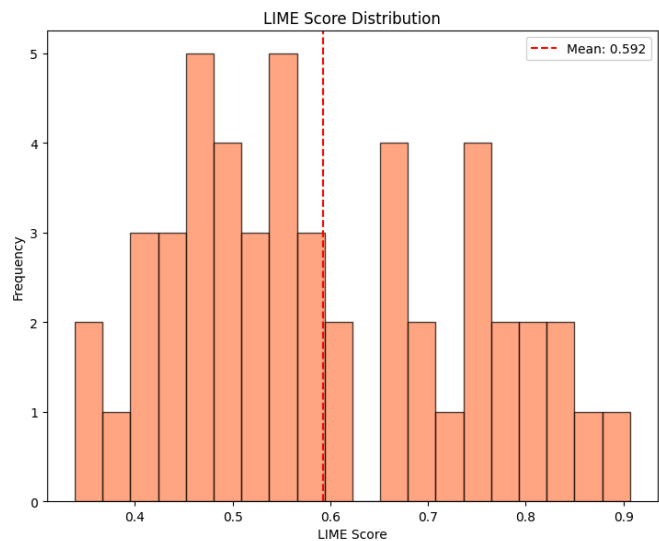
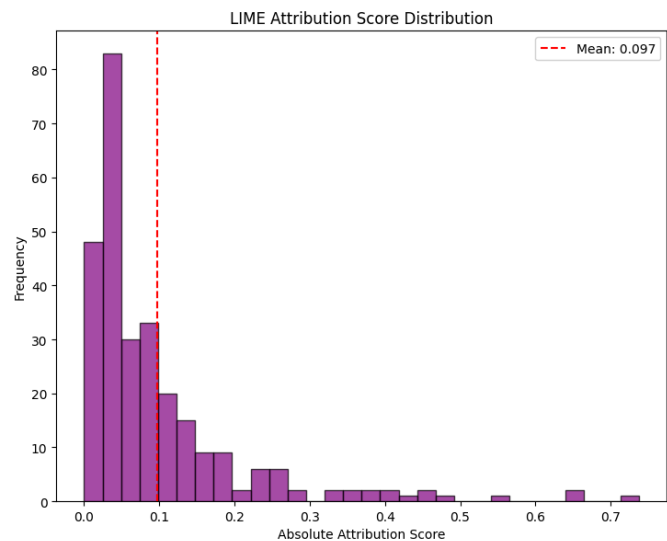
# USAGE EXAMPLES:
#
# Basic run:
# python grace_enhanced_pipeline.py
#
# With custom parameters:
# python grace_enhanced_pipeline.py --num-samples 100 --lime-samples 1500 --chunk-size 32
#
# Force rebuild all cached files:
# python grace_enhanced_pipeline.py --force
#
# Speed vs quality trade-off examples:
# python grace_enhanced_pipeline.py --num-samples 20 --lime-samples 500 # Fast
# python grace_enhanced_pipeline.py --num-samples 100 --lime-samples 2000 # High quality

```

 Starting Enhanced RoBERTa-MNLI + LIME Pipeline for SNLI
 New features: reproducibility, sanity checks, error analysis, stability sweep, comprehensive plots
 All seeds set to 42
 Clearing GPU memory...
 GPU memory cleared.
 Directories setup complete
 Run config saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\run_config.json
 Loading SNLI dataset...
 Loading from local cache...
 SNLI dataset loaded: 550152 training examples
 Sample: {'premise': 'A person on a horse jumps over a broken down airplane.', 'hypothesis': 'A person is training his horse for a competition.', 'label': 1}
 Found existing SNLI sample at c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\sampled_snli_data\snli_sample.json; loading...
 Found existing processed SNLI at c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\processed_snli_data\processed_snli.json; loading...
 Loading roberta-large-mnli model...
 Using device: cuda
 Model loaded successfully!
 Testing model...
 Test input: 'The cat is sleeping on the couch.' vs 'The cat is awake.'
 Probabilities: [0.9956124 0.00212934 0.00225819]
 Predicted: contradiction (confidence: 0.9956)
 Model test passed!
 Found existing LIME explanations at c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\lime_explanations_roberta.json; loading...
 Filtering stopwords from LIME attributions...
 Filtered results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\lime_explanations_filtered.json
 Average attribution reduction: 4.4 tokens
 Computing evaluation metrics (k=3)...
 Computing metrics: 100% ██████████ 50/50 [00:06<00:00, 7.29it/s]
 Performing sanity checks...
 Found 19 correct, 31 incorrect predictions
 Running shuffle sanity test...
 Sanity check results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\sanity_check_results.json
 Performing error analysis...
 Error analysis saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\error_analysis.json
 Running LIME stability mini-sweep...
 Testing params: {'num_samples': 500, 'kernel_width': None} (kw=default)
 Testing params: {'num_samples': 1000, 'kernel_width': None} (kw=default)
 Testing params: {'num_samples': 500, 'kernel_width': 25} (kw=25)
 Testing params: {'num_samples': 1000, 'kernel_width': 25} (kw=25)
 Stability sweep results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\lime_stability_sweep.json
 Enhanced results analysis...

 FINAL EVALUATION RESULTS:
 Model Accuracy: 0.380
 Average Confidence: 0.943
 Average Faithfulness: 0.171 ± 0.163
 Average Sufficiency: 0.538 ± 0.124
 Average Comprehensiveness: 0.122 ± 0.203
 Complete results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\complete_evaluation_results.json
 Creating comprehensive visualization plots...





✓ All plots saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\plots
📄 Generating SNLI LIME report...
⚙️ Computing bootstrap confidence intervals...
✓ Bootstrap confidence intervals computed
✓ Report saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\SNLI_LIME_report.md

🎉 Enhanced pipeline completed successfully!

📁 Generated files:

📄 Results: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\complete_evaluation_results.json

🔧 Config: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\run_config.json

🔍 Sanity: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\sanity_check_results.json

✖ Errors: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\error_analysis.json

📊 Stability: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\lime_stability_sweep.json

📄 Report: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\SNLI_LIME_report.md

📄 Plots: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\plots/